

NYC MTA Subway Hourly Ridership Prediction for Hotspot Transportation Hub Stations

Elijah McCauley, Osborn Wang, Zhixin Wang

May 8, 2026

1 Problems

The New York City subway system is the circulatory system of the city, handling millions of riders daily. However, station crowding and passenger volume are highly volatile. While general rush-hour trends are predictable, actual hourly ridership at specific stations is heavily influenced by exogenous factors such as sudden weather shifts, localized events, and temporal anomalies; a sudden rainstorm, a heatwave, or a weekend disruption can instantly drive thousands of unexpected riders underground, overwhelming specific stations. Currently, station staffing, security deployment, and supplementary transit rebalancing (such as bus availability) often operate on static schedules. This reactive approach leads to severe shortcomings, safety hazards during unexpected surges, and inefficient resource allocation during quiet periods. In the context of urban public policy, accurately forecasting station-level volume is critical for dynamic resource management. This project proposes a predictive analytics approach to model hourly MTA station ridership. By leveraging machine learning techniques to map the complex, non-linear relationships between external factors like precipitation, temperature, time of day and some specific hotspot station volume (Time Square, WTC, Herald Square, etc.), we aim to demonstrate how data-driven forecasting can optimize urban infrastructure, improve commuter experience, and enhance the overall efficiency of public transit operations. Solving this problem is highly impactful for urban planning, as accurate predictive models could allow city agencies to proactively manage staff control, reroute supplementary transit, and significantly improve the daily flow of the city.

2 Related Work and Challenges

2.1 Related Work

2.1.1 Traditional Time Series Approaches

Early work on transit ridership forecasting relied heavily on classical statistical time series models. Autoregressive Integrated Moving Average (ARIMA) and its seasonal variant SARIMA have been widely applied to short-term passenger flow prediction, capturing autocorrelation and periodic structure in ridership data [Sun et al., 2020]. However, these linear models often struggle to capture the multiple over-

lapping seasonalities and sharp non-linearities characteristic of urban subway ridership — particularly the abrupt transitions between overnight lows and rush-hour peaks. Prophet, a decomposable additive model developed by Taylor and Letham [2018], addresses some of these limitations by explicitly modeling trend, multiple seasonal components, and holiday effects as separate interpretable signals, and has seen increasing adoption in applied forecasting contexts where interpretability and ease of use are important.

2.1.2 Machine Learning Approaches

Tree-based ensemble methods have emerged as strong baselines for structured tabular ridership data. Zhou et al. [2026] apply Linear Regression, Decision Tree, and Random Forest models to daily MTA New York City subway ridership from 2022 to 2024, incorporating temporal features (day of week, month, lagged ridership) alongside weather and air quality covariates. Their results show that Random Forest achieves the highest accuracy among the three, and that temporal information and selected weather variables are the most predictive features, while pollution metrics contribute little. Their study is closely related to ours in its use of the same MTA dataset and feature set; however, it operates at daily rather than hourly resolution, models the system as a whole rather than at the individual station level, and does not benchmark against deep learning architectures.

A comparative study of ten statistical and machine learning models on the MBTA system found that temporal calendar features such as day-of-week and season substantially improve predictive accuracy, while weather variables provide more limited and inconsistent gains [Nalamalpu et al., 2025]. Montréal smart card data has similarly been used to benchmark regression and machine learning forecasting methods at quarter-hour resolution, illustrating the challenges associated with fine-grained passenger demand prediction in highly dynamic transit systems [Toqué et al., 2020].

2.1.3 Deep Learning Approaches

Long Short-Term Memory (LSTM) networks, which maintain a cell state across timesteps, are a natural fit for sequential ridership data and have been applied to passenger flow forecasting in several urban metro systems [Zhang et al., 2024, Khalil et al., 2021]. Park et al. [2022] propose a two-step procedure for Seoul Metro that first clus-

ters stations by their functional ridership profiles, then applies time series prediction within each cluster — a finding that motivates our own station-level disaggregation across Manhattan hubs. More sophisticated hybrid architectures such as Convolutional Neural Network (CNN)-LSTM and graph-neural-network-based models have demonstrated further gains by jointly modeling spatial correlations between nearby stations and temporal autocorrelation within each station’s series [Wang and Shalaby, 2024, Khalil et al., 2021]. However, these approaches often depend on explicit network topology information and typically introduce substantially greater computational complexity than the feedforward and recurrent models evaluated in this study.

2.1.4 Weather and Environmental Factors

The influence of weather on transit ridership has been studied extensively. Singhal et al. [2014] use hourly MTA ridership data from 2010–2011 to model weather effects at the station level, demonstrating that adverse weather has a significantly different impact on ridership depending on the time of day, day of week, and station characteristics such as weather protection and connectivity. Their finding that hourly models reveal dynamics invisible in daily aggregations directly motivates our hourly modeling resolution. A more recent study by Guo et al. [2026] develops a vine copula framework to assess NYC subway ridership resilience under extreme weather, capturing non-linear and asymmetric inter-station dependencies. Their results show that heavy precipitation produces the most severe peak-hour impacts while extreme cold primarily reduces off-peak ridership — a finding that informs our inclusion of precipitation, temperature, and wind speed as exogenous features. In contrast to their probabilistic distributional approach, our study focuses on point prediction using supervised machine learning models. The broader literature on metro weather sensitivity similarly suggests that, in many contexts, while weather features can improve model fit, temporal and autoregressive variables often account for a larger share of predictive performance [Hu et al., 2022] — consistent with our SHAP (SHapley Additive exPlanations) analysis showing near-zero weather feature importance in the tree-based model.

2.1.5 Positioning of This Work

The present study occupies a distinct position in this literature. Relative to Zhou et al. [2026], we operate at hourly rather than daily resolution, disaggregate predictions to six individual Manhattan station hubs, and benchmark three model families — Prophet, tree-based ensembles, and deep learning — against each other on the same dataset. Relative to spatio-temporal graph-based deep learning approaches [Wang and Shalaby, 2024], we emphasize interpretability and controlled model comparison rather than architectural complexity, using SHAP analysis to attribute predictions to specific features. Relative to the weather resilience literature [Guo et al., 2026, Singhal et al., 2014], we frame the problem

as operational point forecasting for resource planning rather than probabilistic resilience assessment. The combination of multi-model comparison, station-level hourly granularity, and post-hoc interpretability analysis on the MTA’s 2022–2024 dataset represents a combination of methodological and application-level choices that, to the best of our knowledge, have not been jointly considered in the cited prior work.

2.2 Challenges

Hourly subway ridership is time series data collected over a continuous time period, involving complex, non-linear relationships that make accurate forecasting challenging. The first structural challenge is the heterogeneity across the six hubs. For example, stations like Times Square and Grand Central serve vastly different passenger profiles compared to more localized hubs like Fulton Center, introducing the difficulty of capturing both shared system-wide dynamics and station-specific ridership behaviors within the modeling framework. A second challenge is non-stationarity. The time series may exhibit gradual trend shifts, such as the slow post-pandemic ridership recovery throughout 2022–2024. This means models trained on earlier periods may not generalize well to later ones as behavioral patterns continue to shift. Third, the data contains multiple overlapping seasonalities and strong autocorrelation: ridership exhibits regular daily peaks during morning and evening rush hours, day-of-week patterns contrasting weekday commuter flows with weekend leisure travel, and annual cycles driven by holidays and major events. For instance, ridership in a given hour is strongly correlated with ridership at the same hour the previous day, or the same hour one week prior. The temporal dependencies bring another difficulty in avoiding systematic forecasting errors.

3 Data

This study utilizes the MTA Subway Hourly Ridership dataset from Kaggle, which was originally provided by Metropolitan Transportation Authority (MTA) in New York State. The dataset provides subway ridership entries on an hourly basis at each turnstile by subway station complex and class of fare payment, covering the time period from May 2022 to May 2024. To focus on high-traffic transit nodes, eight station complex IDs were selected from six major Manhattan transportation hubs: World Trade Center, Fulton Center, Grand Central–42nd Street, Penn Station–34th Street, Times Square–42nd Street, and Herald Square–34th Street. These stations represent some of the busiest ridership corridors in the subway system, capturing a diverse mix of commuters, tourists, and inter-modal transfer traffic.

The raw dataset records each observation as a unique combination of station complex ID, fare payment class, and transit timestamp of a one-hour window. Since the fare payment class is not expected to be directly informative for rider-

ship volume prediction, and some transportation hubs span more than one station complex ID, a two-step aggregation was applied. First, ridership counts were grouped by transit timestamp and station complex ID to remove the fare payment dimension. Then, records belonging to the same transportation hub but labeled as different station complex IDs were combined. The resulting dataset thus contains 105,840 rows in total across all six defined transportation hubs. Each row represents the total ridership within a one-hour window at one of the hubs.

To account for the influence of environmental conditions on passenger volume, meteorological data was integrated into the dataset. Historical hourly weather metrics—specifically ambient temperature (`temperature_2m`), precipitation, and wind speed (`wind_speed_10m`)—were retrieved using the Open-Meteo Historical Weather API, localized to the geographical coordinates of Central Manhattan. This external weather data was subsequently joined to the aggregated MTA dataset using a temporal left-merge on the hourly timestamp. This exact temporal alignment ensured that every one-hour subway ridership window was perfectly synchronized with the exact meteorological conditions occurring during that specific hour. Additionally, statutory New York State and US public holidays were programmatically flagged utilizing the Python `holidays` library to capture predictable, system-wide drops in commuter volume.

Following the data aggregation and subsequent feature engineering (detailed in Section 4.2), the final predictive matrix consists of 105,840 observations and 38 features. The target variable to be predicted is the continuous hourly ridership count. The 38 input features comprise exogenous weather metrics, autoregressive lags (1-hour and 24-hour), sine/cosine cyclical time encodings to preserve chronological adjacency, and one-hot encoded station identifiers and statutory holidays.

4 Methods

4.1 Time Series Models

4.1.1 Data Preparation & Exploratory Data Analysis

The dataset was split into individual station-based dataframes, each retaining only the timestamp and ridership columns. Then descriptive statistics of hourly ridership were computed for each station. Specifically, the coefficient of variation (CV, defined as standard deviation divided by mean) was used to measure how far the ridership fluctuations are relative to each station’s average level. To visually confirm the temporal structure, hourly ridership for each station was plotted over a representative 7-month window (October 2022 – April 2023), with each Monday and the first day of each month marked with dashed lines.

4.1.2 Train-Test Split

Each station’s time series was split chronologically to avoid data leakage: the first 80% of observations form the training set (May 15, 2022 – December 25, 2023; 14,112 rows per station), and the remaining 20% form the held-out test set (December 26, 2023 – May 20, 2024; 3,528 rows per station).

4.1.3 Prophet Modeling and Evaluation

The Prophet model was selected for hourly ridership forecasting due to its capability of handling multi-seasonality time series and holiday effects, as well as its component interpretability. Across all stations, the fixed hyperparameter configuration includes: `seasonality_mode='multiplicative'` keeps ridership waves scale proportionally with the overall ridership volume. `changepoint_prior_scale=0.05` allows the trend to shift gradually over the training period. `seasonality_prior_scale=10.0` allows the seasonality components to fit the complex daily and weekly patterns shown in the exploratory plots. `changepoint_range=0.8` makes sure changepoints are only detected within the first 80% of the training window, preventing the model from fitting trend shifts near the end of training. US public holidays were incorporated via `add_country_holidays(country_name='US')`.

Prophet estimates seasonality as a Fourier series. The Fourier orders were tuned for each model by grid search to accurately capture the temporal patterns. The `daily_seasonality` and `weekly_seasonality` were searched over 10, 15, and 20. The `yearly_seasonality` was searched over 5, 10, 15. Higher Fourier orders allow the model to capture sharper, more complex seasonal shapes, while lower orders produce smoother curves.

Each candidate configuration was evaluated using Prophet’s rolling-origin cross-validation (CV), applied entirely within the training set. The CV parameters were defined as: `initial='365 days'` allows that each fold starts with at least one full year of training history, ensuring the model has seen enough annual seasonality. `period='30 days'` set the cutoff date to move forward by 30 days between folds, creating 7 folds in total within the training window. `horizon='30 days'` makes each fold to forecast 30 days ahead. All horizon-window predictions across the 7 folds are pooled to compute the final CV error metrics. The configuration with the lowest Weighted Mean Absolute Percentage Error (WMAPE) was selected as the winner, making the model robust to the near-zero overnight hours that would otherwise inflate standard MAPE. For a station with actual ridership y_t and predicted ridership $\hat{y}_t$, WMAPE was defined as:

$$\text{WMAPE} = \frac{\sum_{t=1}^T |y_t - \hat{y}_t|}{\sum_{t=1}^T |y_t|} \times 100\%.$$

Negative forecasts were clipped to zero before computing

the error metrics. The final model was refit on the full 80% training set using the best parameters.

For each hub, the ridership predictions were decomposed into separate signals for visual inspection, using a future dataframe that extends through the entire training and test window. The decomposition reveals five components: trend, holiday effects, daily seasonality, weekly seasonality, and yearly seasonality.

4.2 Tree-based Models

4.2.1 Feature Engineering and Motivation

Standard machine learning models struggle with the cyclical nature of time. To preserve chronological adjacency, time and day-of-week variables were mapped onto continuous circles using sine and cosine transformations. To motivate this approach, without cyclical encoding, a model would incorrectly assume that 23:00 and 00:00 are 23 hours apart, rather than 1 hour apart, severely distorting late-night predictions.

Furthermore, to provide the models with necessary short-term and routine memory, autoregressive lagged features were engineered. A 1-hour lag (`lag_1h`) was explicitly chosen to capture immediate crowd momentum, and a 24-hour lag (`lag_24h`) was chosen to capture the repeating daily commuter routine. Additional exogenous features included a binary indicator for statutory New York State holidays. Categorical variables were transformed using One-Hot Encoding, and continuous variables were standardized using a `StandardScaler`.

4.2.2 Model Selection and Motivation

To rigorously prevent data leakage across the entire matrix, a strict chronological split was enforced (Train: May 2022–December 2023; Test: January 2024–May 2024). Three distinct models were developed, each motivated by a specific mathematical advantage for tabular time-series data:

- **Random Forest Regressor:** Chosen as the primary baseline. Its bagging technique (training hundreds of independent trees on random subsets of data) is inherently resistant to overfitting and highly robust to the extreme outliers frequently found in public transit data (e.g., sudden event crowds).
- **Gradient Boosting Regressor (GBM):** Chosen to explore whether sequential error-correction could outperform independent bagging. It was optimized via `RandomizedSearchCV` to find the mathematical optimal capacity (Optimal parameters: `subsample=0.8`, `n_estimators=200`, `max_depth=6`, `learning_rate=0.05`).
- **Voting Regressor Ensemble:** Motivated by the hypothesis that combining a parallel model (Random Forest) with a sequential model (GBM) would cancel out the

individual algorithmic blind spots of each, thereby minimizing the overall variance.

4.3 Deep Learning Models

4.3.1 Architecture

A fully-connected feedforward neural network was implemented in PyTorch using the same 38-feature input matrix constructed for the tree-based models. The architecture follows a “funnel” design, progressively compressing the representation through four layers: `Linear(38→128)`, `Linear(128→64)`, `Linear(64→16)`, and a final `Linear(16→1)` output. Each hidden layer is followed by Batch Normalization, a ReLU activation, and Dropout regularization (rates of 0.4, 0.3, and omitted at the final hidden layer, respectively). Batch Normalization was included to stabilize the distribution of layer activations during training, mitigating internal covariate shift across the wide range of input patterns present in a multi-station dataset. Dropout was included to prevent overfitting to the noisy surge events prevalent in the data.

4.3.2 Training Setup

The network was trained using the Huber loss ($\delta = 1.0$), which is quadratic for small errors and linear for large ones. This loss function was chosen over Mean Squared Error because it is less sensitive to the extreme ridership spikes caused by events at stations like Times Square, preventing a small number of outlier observations from dominating the gradient signal. The Adam optimizer was used with a learning rate of 0.001 and an L2 weight decay of 10^{-4} . A `ReduceLROnPlateau` scheduler halved the learning rate whenever validation loss failed to improve for 5 consecutive epochs. Early stopping with a patience of 15 epochs was applied to recover the best-performing checkpoint, with a maximum of 100 epochs and a batch size of 64. Shuffling was disabled in the `DataLoader` to preserve temporal ordering within each batch.

The continuous features (`temperature_2m`, `precipitation`, `wind_speed_10m`, `lag_1h`, `lag_24h`) were standardized using a `StandardScaler` fit exclusively on the training set. The target ridership variable was similarly standardized; predictions were inverse-transformed before computing evaluation metrics.

4.3.3 LSTM Architecture

To evaluate whether a recurrent architecture could better exploit the sequential structure of ridership data, a Long Short-Term Memory (LSTM) network was implemented as a second deep learning model. The LSTM takes the same 38-feature input matrix as the MLP, reshaped to a sequence of length one to conform to the expected three-dimensional input format (batch, sequence, features). The recurrent core consists of two stacked LSTM layers with a hidden dimension of 128 and inter-layer dropout of 0.3,

followed by a single linear output layer. The same training configuration as the MLP was used: Huber loss ($\delta = 1.0$), Adam optimizer (learning rate 0.001, L2 weight decay 10^{-4}), ReduceLROnPlateau scheduling, and early stopping with patience of 15 epochs. This controlled setup isolates the architectural difference — recurrent vs. feedforward — as the sole variable between the two deep learning models.

4.3.4 Data Split Correction

To ensure each hub contributed equally to both partitions, an independent 80/20 chronological split was performed within each station’s time series before concatenation. A naive global split was found to produce an unrepresentative test set due to station-based row ordering, making the per-station approach necessary.

5 Results and Analysis

5.1 Time Series Modeling Analysis

5.1.1 Descriptive Statistics & Exploratory Analysis

Times Square has the highest average hourly ridership of the six selected stations (4,784 per hour), followed by Penn Station (3,656) and Grand Central (3,368). Despite having the highest average ridership, Times Square shows a relatively lower coefficient of variation, indicating relatively moderate fluctuations compared to other stations. Conversely, stations with lower average ridership tend to exhibit higher coefficients of variation. This inverse relationship between mean ridership and coefficient of variation suggests that each station has a distinct character and seasonal waves that do not simply scale up with overall volume.

The 7-month ridership plots (Appendix Figure 4) reveal two dominant periodic patterns: a daily cycle with morning and evening rush hours, and a weekly cycle with lower ridership on weekends compared to weekdays. Notably, the final two weeks of each calendar year show an obvious decrease in ridership across all stations, likely reflecting reduced commuting during the year-end holiday period. This motivated the inclusion of a yearly seasonality component in the time series model, even though the dataset spans only two years.

5.1.2 Prophet Modeling Results

Each station’s cross-validation WMAPE and the final held-out test set metrics were summarized in a table (Table 1). Overall, the models perform reasonably well, with test WMAPEs ranging from 14.49% at Times Square to 22.20% at WTC. All stations show an inflation from cross-validation WMAPE to the test WMAPE. This finding indicates one limitation of this study: Because the training set spans only 19 months (May 2022 – December 2023), the cross-validation folds do not include data from late December through mid-May, which is the period covered by the test set. As a result,

the optimal hyperparameters were chosen without exposure to the seasonal window during late winter and early spring. This underscores the need for additional years of data to make cross-validation more seasonally representative.

Penn Station and Times Square generalize better, with gaps of only 0.76% and 1.45% between cross-validation and test WMAPE, respectively, suggesting their ridership patterns are more stable across seasons for the training folds to provide sufficient signal. Fulton Center and Grand Central show moderate and consistent degradation of 1% each. Notably, Herald Square and WTC exhibit relatively large gaps, 7.61% and 3.10%, respectively. This probably suggests seasonal overfitting during hyperparameter tuning. Both stations have commuter-heavy temporal profiles, their early-spring ridership patterns differ from the fall and winter signals captured by the trained models. The inflated RMSE across all stations further suggests the models’ mispredictions in rush hours.

For each hub, the ridership predictions were decomposed into separate signals for visual inspection (Figure 6), using a future dataframe that extends through the entire training and test window. The decomposition reveals five components: trend, holiday effects, daily seasonality, weekly seasonality, and yearly seasonality.

Table 1: Metrics Comparison of Prophet Cross-Validation vs. Test Set

Hub Station	CV WMAPE (%)	Test MAE	Test RMSE	Test WMAPE (%)
Fulton Center	18.86	419.08	666.01	20.04
Grand Central	19.10	745.32	1136.09	20.09
Herald Square	13.81	584.67	816.59	21.42
Penn Station	17.65	706.62	1038.22	18.41
Times Sq	13.04	734.64	1023.32	14.49
WTC	19.10	414.02	643.30	22.20

5.1.3 Prophet Component Decomposition

All six stations show a general upward trend in ridership throughout 2022–2024, reflecting the post-pandemic recovery. The holiday component shows negative effects on ridership for most US holidays. The largest decreases show around Thanksgiving, Christmas/New Year, and Independence Day, with the magnitude varies by station. The weekly component reveals two distinct behavioral types among the stations. Commuter-heavy stations, such as WTC and Fulton Station, show higher peaks from Monday through Friday and a pronounced trough on weekends. Tourism-oriented stations, such as Times Square, remains relatively flatter with ridership across the whole week. The annual component reveals lower ridership during winter months, with a gradual increase into spring and summer. The daily component shows the classic dual-peak commuter pattern: a morning rush between approximately 8–9 AM and an evening rush between approximately 5–6 PM. The multiple smaller humps shown in some of the daily curves reflect the broader spread of arrival times across different commuter groups.

5.2 Tree-based Models Results

Model performance was evaluated using Mean Absolute Error (MAE) on the 2024 holdout test set. MAE was selected over Mean Squared Error (MSE) because it provides a highly interpretable real-world metric: the average number of passengers by which the prediction deviates from actual turnstile counts.

As shown in Table 2, the Voting Regressor ensemble achieved the highest accuracy, successfully reducing the error baseline established by the standalone models.

Table 2: Tree-Based Model Performance on 2024 Test Set

Model Architecture	MAE	RMSE	WMAPE (%)
Random Forest (Baseline)	212.47	440.16	6.58
Gradient Boosting (Tuned)	227.46	407.77	7.04
Voting Regressor (Ensemble)	211.04	408.13	6.53

High-Level Takeaway: The ensemble approach proved the most effective, demonstrating that MTA ridership can be predicted to within a roughly 211-passenger margin of error per hour. Furthermore, the Random Forest outperforming the tuned Gradient Booster indicates that MTA crowd data is highly noisy; the variance-reducing properties of bagging (Random Forest) are better suited to this dataset than the bias-reducing properties of boosting (GBM).

5.3 Deep Learning Results

5.3.1 Model Variants and Ablations

Three configurations were evaluated to arrive at the final model. The initial naive global split produced an MAE of 861.1 riders and a WMAPE of 37.6%, but this result was invalidated upon discovering the station-leakage bug described in Section 4.3. A second experiment applied a $\log_{1p}$ transformation to the target variable prior to standardization, motivated by the highly right-skewed distribution of ridership. This variant performed substantially worse (MAE: 2,967; WMAPE: 47.1%), likely because the double transformation (log followed by standardization) distorted the gradient landscape during early training and amplified inverse-transformation errors for high-volume stations. The final model used the corrected per-station chronological split with a standardized (non-log) target, yielding a test MAE of 710.4 riders, RMSE of 1,232.7, and WMAPE of 22.1%.

5.3.2 Per-Station Performance

Table 3 reports per-station metrics for the final model on the held-out test set. Lower-volume stations such as Fulton Center (MAE: 285, WMAPE: 13.6%) and WTC (MAE: 307, WMAPE: 16.5%) are predicted most accurately in absolute terms. High-volume hubs—particularly Times Square (MAE: 1,480, WMAPE: 29.2%) and Penn Station (MAE: 923, WMAPE: 24.1%)—exhibit the largest errors. This pattern mirrors the findings from the tree-based models and

reflects the greater magnitude and unpredictability of crowd surges at major transit interchange hubs.

Table 3: Deep Learning Model Per-Station Performance on Test Set

Hub Station	MAE (riders)	WMAPE (%)
Fulton Center	285.4	13.6
Herald Square	427.8	15.7
WTC	307.0	16.5
Grand Central	839.1	22.6
Penn Station	923.3	24.1
Times Square	1479.9	29.2
Overall	710.4	22.1

5.3.3 Residual Analysis by Hour

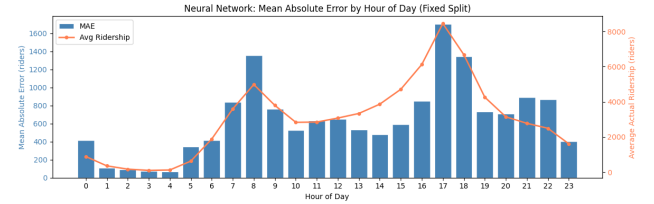


Figure 1: Mean Absolute Error of the neural network overlaid with average actual ridership, by Hour of Day (fixed per-station split).

Figure 1 overlays the model’s mean absolute error against average actual ridership by hour. During overnight hours (1:00–4:00 AM), both ridership and error approach zero, confirming the model performs well on steady-state low-volume periods. The morning commute at hour 8 produces a sharp secondary error spike (~1,340 riders), coinciding with a local ridership peak driven by staggered arrivals across the six stations. The most significant failure mode occurs at hour 17, where the error peak of approximately 1,680 riders coincides precisely with the ridership peak, indicating the model cannot reliably predict the magnitude of the synchronized evening exodus. Signed residual analysis confirms the model is a systematic under-predictor across nearly all hours ($\text{pred} - \text{actual} < 0$), with the most severe under-prediction at hours 8 (−1,174 riders) and 17 (−1,591 riders). This reflects a regression-to-the-mean effect: the model’s weights are optimized across all hours, causing it to anchor predictions toward typical volume and systematically underestimate the magnitude of peak surges.

5.3.4 LSTM Performance

The LSTM model ran for the full 100 epochs without triggering early stopping, with validation loss declining steadily from 0.061 at epoch 5 to 0.016 at epoch 100, indicating continued learning without overfitting. The final model achieved a test MAE of 310.4 riders, RMSE of 511.1, and WMAPE of 9.65% — reducing the MLP’s MAE by 56%

while using the same features and training procedure. This substantial improvement demonstrates that the recurrent cell state allows the LSTM to exploit temporal dependencies in the data more effectively than the feedforward architecture, even with a sequence length of one. The LSTM remains behind the tree-based Voting Regressor (MAE: 211 riders), but narrows the gap considerably compared to the MLP.

5.3.5 Comparison with Tree-Based Models

The neural network’s overall MAE of 710.4 riders is substantially higher than the Voting Regressor ensemble’s MAE of 211 riders. This result is consistent with the broader machine learning literature on structured tabular data, where gradient-boosted trees and random forests frequently outperform deep feedforward networks. The primary driver is the dominance of autoregressive features (`lag_1h`, `lag_24h`): decision trees can partition on these features with sharp, exact thresholds that directly encode the "current volume $\approx$ recent volume" heuristic, whereas an MLP must learn this relationship implicitly through matrix multiplications across several layers. Furthermore, the Dropout regularization introduced to protect against event-surge overfitting may also suppress the network’s ability to fully exploit the autoregressive signal during inference, broadening the prediction distribution at the cost of peak accuracy. The LSTM’s substantially lower error (MAE: 310.4) relative to the MLP (MAE: 710.4) confirms that recurrent architectures are better suited to this sequential domain than feedforward networks, even when the sequence length is set to one. However, the tree-based ensemble still outperforms both deep learning models, reinforcing that sharp autoregressive thresholds learned by decision trees remain the most effective mechanism for tabular transit data.

5.4 Evaluation Comparison

5.4.1 Overall Architecture Performance

To rigorously evaluate the efficacy of the three distinct methodologies, Table 4 compiles the overarching metrics for the univariate time-series (Prophet), the machine learning ensemble (Voting Regressor), and the deep learning network (MLP). Because the Prophet model was trained on six individual hub-specific splits, its metrics are reported as the unweighted macro-average across all six stations to facilitate a fair comparison against the globally trained tree and neural architectures.

Table 4: Comprehensive Evaluation of All Forecasting Architectures

Model Architecture	MAE	RMSE	WMAPE (%)
Prophet (Macro-Average)*	600.73	887.26	19.44
Random Forest (Baseline)	212.47	440.16	6.58
Gradient Boosting (Tuned)	227.46	407.77	7.04
Voting Regressor (Tree Ensemble)	211.04	408.13	6.53
Deep Learning (MLP)	710.40	1232.70	22.10
LSTM	310.40	511.10	9.65

*Prophet metrics represent the average across the six individual hub models.

The consolidated results starkly highlight the superiority of the tree-based methodologies for this specific tabular dataset. The Voting Regressor ensemble achieved an exceptionally low WMAPE of 6.53%, significantly outperforming both the Prophet models and the Deep Learning architectures. The results also reveal an important architectural distinction within the deep learning family: the LSTM (WMAPE: 9.65%) substantially outperforms the feedforward MLP (WMAPE: 22.10%), demonstrating that recurrent architectures are meaningfully better suited to sequential ridership data. The MLP’s high RMSE (1,232.70) further confirms its vulnerability to peak surge events, a weakness the LSTM partially mitigates through its cell-state memory.

5.4.2 Feature Importance (SHAP)

To provide an in-depth analysis of the model’s decision-making process, game-theoretic SHAP (SHapley Additive exPlanations) values were extracted (Figure 2). The analysis revealed three distinct tiers of logic: First, autoregression dominated: the `lag_1h` feature overwhelmingly dictated prediction volume, proving that short-term momentum is the strongest indicator of current volume. Second, cyclical temporal features (`hour_sin`, `hour_cos`) successfully anchored the predictions to routine daily commute schedules. Third, exogenous weather features (precipitation and temperature) had near-zero impact on the model’s final output. This negative result is highly informative: it demonstrates that macro-level transit volume in New York City is highly inelastic to weather conditions.

5.4.3 Residual Analysis: Misclassified Datapoints

To understand exactly which datapoints the models fail to capture accurately, an hourly analysis of the ensemble model’s absolute residuals was conducted (Figure 3).

The analysis reveals that the model performs exceptionally well on "steady-state" datapoints. During off-peak hours (1:00 AM to 4:00 AM), where passenger flow is sparse and consistent, prediction errors drop to near zero.

Conversely, the model systematically "misclassifies" (under-predicts or over-predicts) datapoints during periods of extreme structural volatility. A secondary error peak occurs during the morning commute at 8:00 AM (~ 360 riders).



Figure 2: SHAP Summary Plot detailing feature impact on model output.

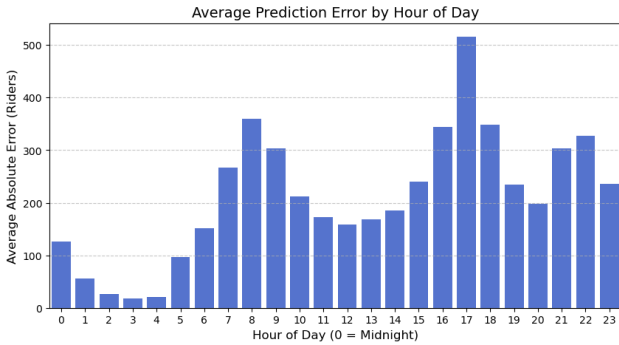


Figure 3: Average Absolute Error of the ensemble model by Hour of Day.

However, the most significant failure mode occurs on data-points belonging to exactly 5:00 PM (Hour 17), with errors exceeding 500 riders.

This discrepancy highlights a limitation in the data architecture itself. The morning commute is somewhat staggered (spread between 7:30 AM and 9:30 AM), allowing the model to capture the volume with moderate accuracy. In contrast, the 5:00 PM evening rush represents a massive, synchronized exodus from office buildings. Because Tree-based models calculate predictions based on the mean of historical leaf nodes, they inherently struggle to predict extreme, sudden outliers. Furthermore, an hourly boundary is too wide to properly capture a crowd surge that may happen entirely between 5:00 PM and 5:15 PM. Future iterations should explore 15-minute interval data to better isolate these concentrated surges.

6 Summary

This study aimed to accurately forecast highly volatile, hourly subway ridership across six major Manhattan transportation hubs to facilitate proactive, dynamic urban resource management. To capture the complex, non-linear dynamics of commuter behavior, we evaluated three distinct predictive architectures: univariate time-series forecasting (Prophet), tree-based machine learning (Random Forest, Gradient Boosting, Voting Regressor), and deep learning (Feedforward MLP and LSTM Network).

The empirical results demonstrated that the tree-based Voting Regressor ensemble achieved the highest predictive accuracy, establishing a robust baseline error of just 211.04 riders per hour (MAE). By fusing the variance-reducing bagging of the Random Forest with the sequential error-correction of the Gradient Booster, the ensemble successfully managed the noisy, surge-heavy nature of transit data. Prophet provided excellent interpretability for macro-level seasonalities and holiday effects, but exhibited seasonal overfitting and struggled to pinpoint the magnitude of daily rush-hour spikes. The deep learning results revealed an important architectural distinction. The feedforward MLP yielded a negative result relative to the tree-based baseline, with an MAE of 710.4 riders, underscoring a critical limitation of non-recurrent architectures on tabular sequential data: decision trees can establish sharp, exact thresholds for autoregressive features that directly encode the current-volume-mirrors-recent-volume heuristic, whereas the MLP must learn this implicitly. However, the LSTM recovered much of this gap, achieving an MAE of 310.4 riders by leveraging its recurrent cell state to more effectively exploit temporal dependencies. This contrast between the two deep learning architectures is itself a meaningful finding: for transit ridership forecasting, recurrence matters more than depth.

In-depth SHAP and residual analyses yielded three critical high-level takeaways regarding urban transit dynamics. First, ridership is overwhelmingly autoregressive; short-term momentum (one-hour lag) and daily cyclical routines are the primary drivers of hub volume, indicating that crowds inherently beget crowds. Second, macro-level subway ridership in New York City is highly inelastic to environmental factors; exogenous weather variables like precipitation and temperature had near-zero impact on commuter volume. This sociological insight proves that the subway is a strict necessity for the urban workforce; the city cannot rely on inclement weather to naturally suppress platform crowding. Finally, structural volatility remains the primary failure mode for all models. While predictions were exceptionally accurate during steady-state hours (1:00 AM to 4:00 AM), the models systematically misclassified the peak 5:00 PM mass exodus.

This specific failure mode exposes an asymmetric reality of the daily commute: while morning arrivals are staggered between 7:30 AM and 9:30 AM, the evening departure is a mas-

sive, synchronized event. Ultimately, this research proves that data-driven forecasting can effectively predict baseline transit volume to optimize staffing, direct turnstile flow, and rebalance supplementary transit. However, because the 5:00 PM commuter wave is so violently concentrated, an hourly temporal boundary is mathematically too wide to capture its peak variance. Future work should transition from hourly data to 15-minute interval forecasting to properly isolate, predict, and proactively manage the concentrated dynamics of extreme crowd surges.

References

- Yan Guo, Brian Yueshuai He, Joseph Y. J. Chow, Zhiya Su, and Omar Wani. Assessing subway ridership resilience under extreme weather with vine copula modeling. *npj Sustainable Mobility and Transport*, 3:25, 2026.
- Yiming Hu, Yangchuan Huang, Shuying Liu, Yuanyang Qi, and Danhui Bai. Exploring the impact of weather on Metro demand forecasting using machine learning method. arXiv preprint arXiv:2210.13965, 2022.
- Sergey Khalil et al. Forecasting public transport ridership: Management of information systems using CNN and LSTM architectures. In *Procedia Computer Science*, volume 184, pages 283–290. Elsevier, 2021. doi: 10.1016/j.procs.2021.03.036.
- Sai Siddharth Nalamalpu, Kaining Yuan, Aiden Zhou, and Eugene Pinsky. Forecasting MBTA transit dynamics: A performance benchmarking of statistical and machine learning models. arXiv preprint arXiv:2512.02336, 2025.
- Yujin Park, Yoonhee Choi, Kyongwon Kim, and Jae Keun Yoo. Machine learning approach for study on subway passenger flow. *Scientific Reports*, 12:2754, 2022. doi: 10.1038/s41598-022-06767-7.
- Abhishek Singhal, Camille Kamga, and Anil Yazici. Impact of weather on urban transit ridership. *Transportation Research Part A: Policy and Practice*, 69:379–391, 2014. doi: 10.1016/j.tra.2014.09.008.
- Shaolong Sun, Dongchuan Yang, Ju-e Guo, and Shouyang Wang. AdaEnsemble learning approach for metro passenger flow forecasting. arXiv preprint arXiv:2002.07575, 2020.
- Sean J. Taylor and Benjamin Letham. Forecasting at scale. *The American Statistician*, 72(1):37–45, 2018.
- Florian Toqué, Etienne Côme, et al. Forecasting of the Montréal subway smart card entry logs with event data. arXiv preprint arXiv:2008.09842, 2020.
- Jiahao Wang and Amer Shalaby. DST-TransitNet: A dynamic spatio-temporal deep learning model for scalable and efficient network-wide prediction of station-level transit ridership. arXiv preprint arXiv:2410.15013, 2024. Accepted at TRB 2025.
- Xin Zhang et al. Short-term passenger flow prediction for urban rail systems: A deep learning approach utilizing multi-source big data. *PLOS ONE*, 2024. doi: 10.1371/journal.pone.0333094.
- Leheng Zhou, Zehao Yang, Wanieong Zheng, Sos S. Agaian, and Xiangjie Kong. Predicting subway ridership using machine learning and environmental factors. In *Proceedings of SPIE, the international society for optical engineering*, volume 14064, pages 140644Q–140644Q–9. SPIE, 2026. ISBN 9798902320876.

Appendix: Supplemental Data and Visualizations

Table 5: Summary Statistics of Hourly Ridership for All Stations

Station	Count	Mean	Std	Min	25%	Median	75%	Max	CV
Fulton Center	17,640	1,991.65	2,048.31	0	422.0	1,633.0	2,611.3	13,866	1.028
Grand Central	17,640	3,368.20	3,340.88	0	789.0	2,754.0	4,405.0	21,070	0.992
Herald Square	17,640	2,656.40	2,234.40	0	689.0	2,512.0	3,692.3	12,883	0.841
Penn Station	17,640	3,656.32	2,941.15	0	1,160.0	3,573.5	4,859.0	17,215	0.804
Times Square	17,640	4,784.44	3,245.90	0	2,134.0	4,764.0	6,595.0	18,396	0.678
WTC	17,640	1,732.63	1,930.15	0	281.0	1,281.5	2,321.0	13,695	1.114

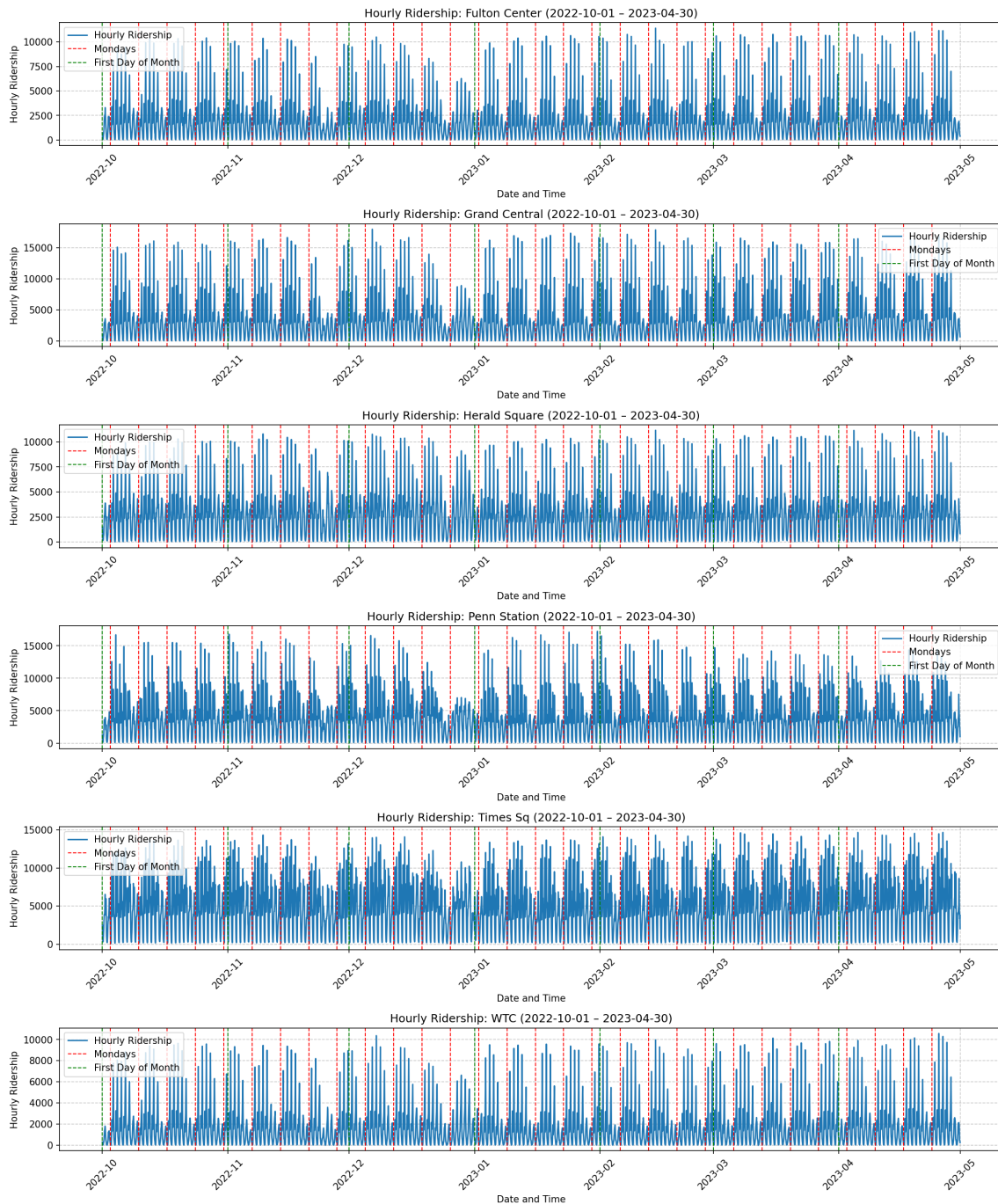


Figure 4: Hourly Ridership for All Stations (Time Window: Oct 2022 - Apr 2023)

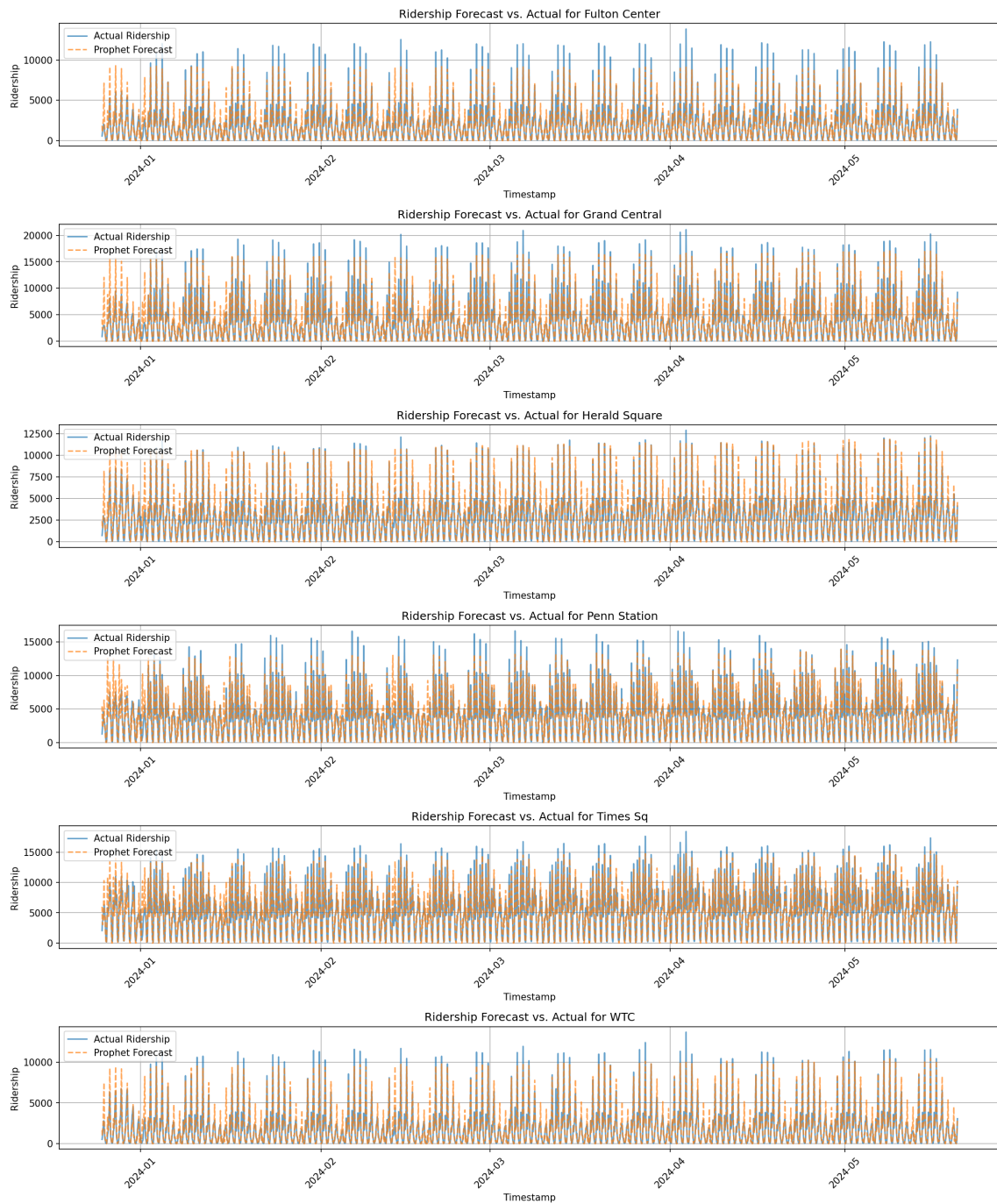
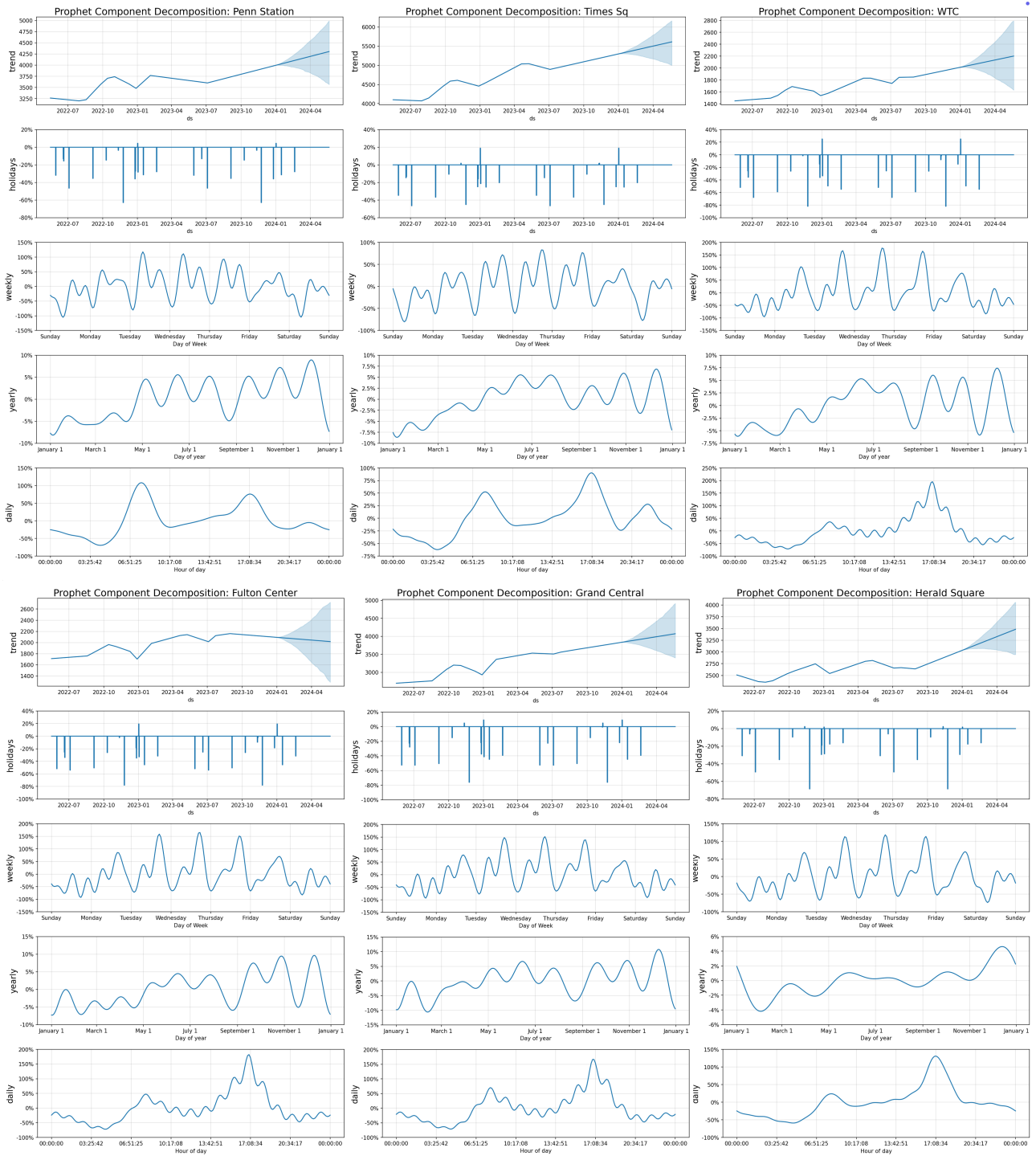


Figure 5: Prophet Forecasting of Hourly Ridership for All Stations (Test Window: late Dec 2023 - May 2024)



1 Code

https://drive.google.com/drive/folders/12Y_DQLjvQB16C5ig4zTK1ohnyhisN5Eq?usp=sharing